Center for Data Analytics and Modeling

Chuka University

Machine Learning Assignment 2

Multi-Model Classification for CVD Prediction

Course: Machine Learning Methods
Instructor: Lumumba W. Victor
Email: lumumbavictor172@gmail.com
Date Issued: March 31, 2026
Due Date: 10th April 2026
Total Marks: 100

Learning Objectives

- Implement multiple classification algorithms in R (Logistic, KNN, SVM, RF)
- Apply appropriate preprocessing for different model types
- Create and use trainControl schemes for cross-validation
- Evaluate and compare model performance using multiple metrics
- Interpret model-specific outputs (coefficients, importance, support vectors)
- Make evidence-based model selection recommendations

Required Packages

```r
library(tidyverse)     # Data manipulation
library(caret)         # Unified ML interface
library(pROC)          # ROC analysis
library(e1071)         # SVM
library(class)         # KNN
library(randomForest)  # Random Forest
```

1 General Instructions

- This assignment must be completed **individually**.
- Submit your work as a **PDF report** and the **R script** (.R or .Rmd).
- All code must be well-commented and reproducible (set seed = 2026).
- Use the provided dataset: `medical_data.csv`.
- Plots and tables must be clearly labeled with captions.
- Late submissions will incur a penalty of 10% per day.
- Plagiarism will result in zero marks for all parties involved.

Important Notes

- The target variable is **CVD** (Cardiovascular Disease), coded as binary (0 = No, 1 = Yes).
- Convert CVD to a factor before modeling: `factor(CVD, levels = c(0,1), labels = c("No", "Yes"))`
- Some models (KNN, SVM) require feature scaling; others (RF) do not.
- Use the **same train-test split** for all models to ensure fair comparison.

2 Dataset Description

The dataset `medical_data.csv` contains health indicators for 2,000 patients. The goal is to predict the presence of Cardiovascular Disease (CVD) using multiple machine learning models.

3 Tasks

3.1 Task 1: Data Import and Exploration (10 Marks)

1. Load all necessary libraries (tidyverse, caret, pROC, e1071, class, randomForest).
2. Import the dataset `medical_data.csv`.
3. Display the structure of the data (`str()`) and summary statistics (`summary()`).
4. Check for missing values and report the count and percentage per variable.
5. Visualize the distribution of the target variable **CVD** using a bar plot.
6. Calculate and report the class imbalance ratio.

Variable	Type	Description
PatientID	Integer	Unique identifier
Age	Numeric	Age in years (18-80)
Gender	Factor	Male / Female
BMI	Numeric	Body Mass Index (16-45)
SBP	Numeric	Systolic Blood Pressure (90-180 mmHg)
DBP	Numeric	Diastolic Blood Pressure (60-110 mmHg)
Cholesterol	Numeric	Total Cholesterol (120-320 mg/dL)
Glucose	Numeric	Fasting Glucose (70-200 mg/dL)
Smoking	Factor	Never / Former / Current / Occasional
Alcohol	Factor	None / Moderate / Heavy
Exercise	Factor	Sedentary / Light / Moderate / Vigorous
FamilyHistory	Factor	No / Yes
Medication	Factor	No / Yes
SleepHours	Numeric	Hours per night (4-10)
StressLevel	Numeric	Scale 1-10
DietQuality	Numeric	Scale 1-5 (5 = best)
CVD	**Binary**	**Target (0 = No, 1 = Yes)**

Table 1: Variable Description for Medical Dataset

Code Hint

```r
library(tidyverse)
library(caret)
library(pROC)

# Load data
data <- read_csv("medical_data.csv")

# Check structure and missing values
str(data)
summary(data)
colSums(is.na(data))

# Target distribution
ggplot(data, aes(x = factor(CVD))) +
  geom_bar(fill = "steelblue") +
  labs(title = "CVD Distribution", x = "CVD", y = "Count")
```

3.2 Task 2: Data Preprocessing (15 Marks)

1. Handle missing values (justify your method: removal or imputation).
2. Convert categorical variables to factors:
   - Gender, Smoking, Alcohol, Exercise, FamilyHistory, Medication
3. Convert the target variable `CVD` to a factor with levels `"No"` (0) and `"Yes"` (1).
4. **For KNN and SVM:** Scale numeric predictors (mean = 0, sd = 1) using `preProcess()` from `caret`.
5. **For Random Forest:** Scaling is not required (tree-based).
6. Document your preprocessing decisions in the report.

Code Hint

```r
# Convert CVD to factor
data$CVD <- factor(data$CVD,
                   levels = c(0, 1),
                   labels = c("No", "Yes"))

# Convert categorical variables to factors
categorical_vars <- c("Gender", "Smoking", "Alcohol",
                      "Exercise", "FamilyHistory", "Medication")
data[categorical_vars] <- lapply(data[categorical_vars], factor)

# Handle missing values (example: remove rows)
data_clean <- na.omit(data)

# For scaling (KNN, SVM only)
numeric_vars <- c("Age", "BMI", "SBP", "DBP", "Cholesterol",
                  "Glucose", "SleepHours", "StressLevel", "DietQuality")
preproc <- preProcess(data_clean[, numeric_vars], method = c("center", "scale"))
data_scaled <- data_clean
data_scaled[, numeric_vars] <- predict(preproc, data_clean[, numeric_vars])
```

3.3 Task 3: Data Partitioning (10 Marks)

1. Set the seed to `2026` for reproducibility.
2. Split the data into 70% training and 30% testing using **stratified sampling** (to preserve class balance).
3. Report the class distribution (counts and percentages) in both training and testing sets.

4. Verify that the class proportions are similar in both sets.

Code Hint

```r
set.seed(2026)

# Stratified split
train_index <- createDataPartition(data_clean$CVD, p = 0.70,
    list = FALSE)
train_data <- data_clean[train_index, ]
test_data <- data_clean[-train_index, ]

# Verify class distribution
cat("Training Set:\n")
print(table(train_data$CVD))
prop.table(table(train_data$CVD))

cat("\nTesting Set:\n")
print(table(test_data$CVD))
prop.table(table(test_data$CVD))
```

3.4 Task 4: TrainControl Scheme (10 Marks)

1. Define a `trainControl` scheme with the following specifications:
   - Method: 10-fold cross-validation
   - Save predictions for analysis
   - Enable class probabilities (for ROC/AUC)
   - Use `twoClassSummary` for performance metrics
2. Explain why cross-validation is important for model evaluation.
3. Save this control object to use across all models.

Code Hint

```r
# Define trainControl
ctrl <- trainControl(
  method = "cv",                # 10-fold cross-validation
  number = 10,                  # 10 folds
  savePredictions = "final",    # Save predictions
  classProbs = TRUE,            # Enable probability predictions
  summaryFunction = twoClassSummary,  # ROC, Sens, Spec
  verboseIter = FALSE           # Suppress iteration output
)

# Explain in report why CV is important
```

3.5 Task 5: Model Training - Logistic Regression (15 Marks)

1. Train a Logistic Regression model using `caret::train()` with method `"glm"`.
2. Alternatively, fit a baseline model using base R `glm()` function.
3. Extract and report the model coefficients.
4. Calculate Odds Ratios (OR) for at least 3 significant predictors: $OR = e^{\beta}$
5. Interpret the Odds Ratios in the context of CVD risk.

Code Hint

```r
# Train Logistic Regression
log_model <- train(
  CVD ~ Age + Gender + BMI + SBP + Cholesterol +
        Glucose + Smoking + FamilyHistory + Exercise,
  data = train_data,
  method = "glm",
  family = binomial,
  trControl = ctrl
)

# View coefficients
summary(log_model$finalModel)$coefficients

# Calculate Odds Ratios
coefficients <- coef(log_model$finalModel)
odds_ratios <- exp(coefficients)
print(round(odds_ratios, 3))
```

3.6 Task 6: Model Training - KNN (15 Marks)

1. Train a K-Nearest Neighbors model using `caret::train()` with method `"knn"`.
2. Tune the parameter `k` (number of neighbors) using the grid: `k = 1, 3, 5, ..., 21`.
3. **Important:** Use the scaled dataset for KNN (distance-based algorithm).
4. Report the optimal k value selected by cross-validation.
5. Plot accuracy vs. k to visualize the tuning results.

Code Hint

```r
# Train KNN (use scaled data)
knn_model <- train(
  CVD ~ Age + Gender + BMI + SBP + Cholesterol +
        Glucose + Smoking + FamilyHistory + Exercise,
  data = train_scaled,  # Use scaled data!
  method = "knn",
  trControl = ctrl,
  tuneGrid = expand.grid(k = seq(1, 21, by = 2))
)

# View optimal k
knn_model$bestTune

# Plot accuracy vs k
plot(knn_model, main = "KNN: Accuracy vs k")
```

3.7 Task 7: Model Training - SVM (15 Marks)

1. Train a Support Vector Machine model using `caret::train()` with method `"svmRadial"`.
2. Tune the hyperparameters:
   - `C` (cost): 0.25, 0.5, 1, 2, 4
   - `sigma`: 0.01, 0.05, 0.1, 0.5
3. **Important:** Use the scaled dataset for SVM.
4. Report the optimal C and sigma values.
5. Explain the role of the kernel function in SVM.

Code Hint

```r
# Train SVM (use scaled data)
svm_model <- train(
  CVD ~ Age + Gender + BMI + SBP + Cholesterol +
        Glucose + Smoking + FamilyHistory + Exercise,
  data = train_scaled,  # Use scaled data!
  method = "svmRadial",
  trControl = ctrl,
  tuneGrid = expand.grid(
    C = c(0.25, 0.5, 1, 2, 4),
    sigma = c(0.01, 0.05, 0.1, 0.5)
  )
)

# View optimal parameters
svm_model$bestTune

# Plot tuning results
plot(svm_model, main = "SVM: ROC vs Parameters")
```

3.8 Task 8: Model Training - Random Forest (15 Marks)

1. Train a Random Forest model using `caret::train()` with method `"rf"`.
2. Tune the parameter `mtry` (features per split): 2, 4, 6, 8.
3. Set `ntree = 500` (number of trees).
4. Report the optimal mtry value.
5. Extract and plot variable importance.
6. Identify the top 5 most important predictors for CVD.

Code Hint

```r
# Train Random Forest (scaling not required)
rf_model <- train(
  CVD ~ Age + Gender + BMI + SBP + Cholesterol +
        Glucose + Smoking + FamilyHistory + Exercise,
  data = train_data,  # Original data (no scaling needed)
  method = "rf",
  trControl = ctrl,
  tuneGrid = expand.grid(mtry = c(2, 4, 6, 8)),
  ntree = 500
)

# View optimal mtry
rf_model$bestTune

# Variable importance
varImp(rf_model)
plot(varImp(rf_model), main = "Random Forest: Variable
    Importance")
```

3.9 Task 9: Model Evaluation and Comparison (20 Marks)

1. Predict the class and probabilities on the **test set** for all 4 models.
2. For each model, generate a Confusion Matrix and report:
   - Accuracy
   - Sensitivity (Recall)
   - Specificity
   - Precision (PPV)
   - F1-Score
   - Kappa
3. Plot ROC curves for all 4 models on the same graph.
4. Calculate and compare AUC values for all models.
5. Create a comparison table summarizing all metrics.
6. Rank the models from best to worst based on AUC.

Code Hint

```r
# Predictions for all models
log_pred <- predict(log_model, test_data)
log_probs <- predict(log_model, test_data, type = "prob")

knn_pred <- predict(knn_model, test_scaled)
knn_probs <- predict(knn_model, test_scaled, type = "prob")

svm_pred <- predict(svm_model, test_scaled)
svm_probs <- predict(svm_model, test_scaled, type = "prob")

rf_pred <- predict(rf_model, test_data)
rf_probs <- predict(rf_model, test_data, type = "prob")

# Confusion Matrix (example for Logistic)
confusionMatrix(log_pred, test_data$CVD, positive = "Yes")

# ROC Curves
log_roc <- roc(test_data$CVD, log_probs$Yes)
knn_roc <- roc(test_data$CVD, knn_probs$Yes)
svm_roc <- roc(test_data$CVD, svm_probs$Yes)
rf_roc <- roc(test_data$CVD, rf_probs$Yes)

# Plot all ROC curves
plot(log_roc, col = "blue", lwd = 2, main = "ROC Comparison")
lines(knn_roc, col = "green", lwd = 2)
lines(svm_roc, col = "red", lwd = 2)
lines(rf_roc, col = "purple", lwd = 2)
legend("bottomright",
       legend = c("Logistic", "KNN", "SVM", "RF"),
       col = c("blue", "green", "red", "purple"),
       lwd = 2)

# AUC Comparison
auc_values <- data.frame(
  Model = c("Logistic", "KNN", "SVM", "Random Forest"),
  AUC = c(as.numeric(auc(log_roc)),
          as.numeric(auc(knn_roc)),
          as.numeric(auc(svm_roc)),
          as.numeric(auc(rf_roc)))
)
print(auc_values)
```

3.10 Task 10: Interpretation and Recommendations (15 Marks)

1. Based on your results, answer the following:
   - Which model performed best? Justify using metrics.
   - Which model performed worst? Explain possible reasons.

- Was feature scaling necessary for all models? Explain.
- Did cross-validation improve model reliability? Explain.

2. For the best model, identify the top 3 risk factors for CVD.
3. Discuss the trade-offs between model accuracy and interpretability.
4. Provide recommendations for healthcare practitioners based on your findings.
5. Discuss any limitations of your analysis (e.g., observational data, potential confounders).

Report Guidelines

Your report should include:

- Executive summary (1 paragraph)
- Methodology description
- Results with tables and figures
- Model comparison table
- Interpretation and recommendations
- Limitations and future work
- Appendix with complete R code

4 Submission Guidelines

- **File Format:** Submit a single ZIP file named `ML_Assignment2_YourName.zip`.
- **Contents:**
  1. Report (PDF) containing answers, plots, tables, and interpretations (max 15 pages).
  2. R Script (.R) or R Markdown (.Rmd) with all code and comments.
  3. The dataset (if modified).
- **Naming Convention:** `YourName_StudentID_ML_Assignment2.pdf`
- **Submission Portal:** Upload to the CDAM Learning Management System.
- **Deadline:** [Insert Date and Time]

Component	**Marks**	**Criteria**
Data Import & Exploration	10	Correct loading, summary, visualization, missing value report
Preprocessing	15	Proper handling of NA, factors, scaling for appropriate models
Data Partitioning	10	Correct 70/30 split, stratification, seed set, distribution report
TrainControl Scheme	10	Correct CV setup, explanation of importance
Logistic Regression	15	Model fitting, coefficients, odds ratios, interpretation
KNN Model	15	Model fitting, k tuning, scaled data usage, optimal k report
SVM Model	15	Model fitting, C/sigma tuning, scaled data usage, kernel explanation
Random Forest	15	Model fitting, mtry tuning, variable importance, top predictors
Model Evaluation	20	Confusion matrices, ROC curves, AUC comparison, metrics table
Interpretation	15	Model ranking, risk factors, recommendations, limitations
Total	**100**	

Table 2: Assignment Grading Rubric

5 Grading Rubric

6 Tips for Success

Best Practices

- **Start Early:** This assignment requires training 4 models; allow time for computation.
- **Use Comments:** Comment your code to explain each step (helps with partial marks).
- **Check Scaling:** Remember KNN and SVM need scaled data; RF and Logistic do not require it.
- **Same Split:** Use the same train-test split for all models to ensure fair comparison.
- **Save Objects:** Save your trained models to avoid re-running if R crashes.
- **Visualize:** Include ROC curves, variable importance plots, and tuning plots.
- **Interpret:** Don't just report numbers; explain what they mean for CVD prediction.
- **Proofread:** Check your report for clarity, grammar, and proper labeling of figures.

Common Mistakes to Avoid

- **Data Leakage:** Do not scale using test data; fit scaler on training data only.
- **Wrong Target Type:** CVD must be a factor, not numeric, for classification.
- **Inconsistent Splits:** Using different splits for different models invalidates comparison.
- **Missing Metrics:** Report multiple metrics (not just accuracy) for imbalanced data.
- **No Interpretation:** Numbers without context earn fewer marks.
- **Uncited Code:** If using online resources, cite them appropriately.

7 Additional Resources

- **caret Documentation:** https://topepo.github.io/caret/
- **R for Data Science:** https://r4ds.had.co.nz/
- **Introduction to Statistical Learning:** https://www.statlearning.com/
- **CDAM Resources:** https://cdam.chuka.ac.ke
- **Office Hours:** Tuesdays 2-4 PM, Science Block Room 205

Center for Data Analytics and Modeling (CDAM)
Chuka University
"Data Driven Decisions for Sustainable Development"
https://cdam.chuka.ac.ke

This assignment is designed to assess practical machine learning skills.
All work must be your own. Academic integrity is strictly enforced.